

КАК РАБОТАЕТ СВЯЗЬ QML ↔ C++ (простым языком)

ТЕОРИЯ

Что такое сигналы и слоты?

Представь рацию:

- **Сигнал** = ты говоришь в рацию "Я нашёл клад!"
- **Слот** = кто-то слушает рацию и реагирует "Иду к тебе!"

В Qt:

```
cpp

// C++ класс испускает сигнал
emit clipAdded(5); // "Я добавил клип №5!"

// QML слушает и реагирует
Connections {
    target: cppTimeline
    function onClipAdded(index) {
        console.log("Получил клип", index) // "Иду к тебе!"
    }
}
```

ТВОЙ СЛУЧАЙ: Timeline

Шаг 1: C++ создаёт объект

```
cpp

// main.cpp
Timeline timeline; // Создали объект в C++
```

Шаг 2: Регистрируем в QML

```
cpp

// Делаем C++ объект видимым для QML
engine.rootContext()->setContextProperty("cppTimeline", &timeline);
//           ↑           ↑
//           Имя в QML   Адрес объекта
```

Теперь в QML доступен объект **cppTimeline**!

Шаг 3: QML вызывает C++ функции

```
qml

// ClipManager.qml
function addClip(filepath, track, time) {
    // Вызываем C++ функцию!
    cppTimeline.addClip(filepath, track, time)
    //   ↑       ↑
    // Объект   Функция помеченная Q_INVOKABLE
}
```

Шаг 4: C++ делает работу

```
cpp

// timeline.cpp
bool Timeline::addClip(const QString& filepath, int trackIndex, double startTime) {
    // 1. Открыть видео через FFmpeg
    double duration = getClipSourceDuration(filepath);

    // 2. Создать клип
    TimelineClip newClip;
    newClip.filePath = filepath;
    newClip.duration = duration; // Реальная длина!

    // 3. Добавить в список
    m_clips.append(newClip);

    // 4. ИСПУСТИТЬ СИГНАЛ в QML!
    emit clipsChanged(); // "Эй, QML! Клипы изменились!"
    emit clipAdded(newIndex); // "Добавлен клип №X!"

    return true;
}
```

Шаг 5: QML слушает сигналы

```
qml
```

```
// ClipManager.qml
Connections {
    target: cppTimeline // Слушаем сигналы от C++ объекта

    function onClipsChanged() {
        console.log("✅ C++ сказал что клипы изменились!")
        updateClips() // Обновляем UI
    }

    function onClipAdded(index) {
        console.log("✅ C++ добавил клип", index)
    }
}
```

🔧 ЧТО НУЖНО ИЗМЕНИТЬ В ТВОЁМ QML

В ClipManager.qml:

qml

```
QObject {
    id: root

    property var clips: []
    property int revision: 0

    // УДАЛИ ВСЮ ЛОГИКУ addClip! Она теперь в C++

    // Вместо этого просто вызывай C++:
    function addClip(filepath, trackNumber, startTime) {
        console.log("📞 QML: Вызываю C++ addClip")

        // Вызов C++ функции
        var success = cppTimeline.addClip(filepath, trackNumber, startTime)

        if (success) {
            console.log("✅ C++ успешно добавил клип!")
        } else {
            console.log("❌ C++ не смог добавить клип")
        }
    }

    // Слушаем сигналы от C++
    Connections {
        target: cppTimeline

        function onClipsChanged() {
            console.log("📡 Получен сигнал: clipsChanged")
            root.updateClipsFromCpp()
        }

        function onClipAdded(index) {
            console.log("📡 Получен сигнал: clipAdded", index)
        }

        function onTotalDurationChanged() {
            console.log("📡 Длительность timeline изменилась")
            // TODO: Обновить playbackManager.duration
        }
    }

    // Функция для синхронизации QML clips с C++
    function updateClipsFromCpp() {
        // TODO: Запросить список клипов из C++
        // Для этого нужна функция getClipsForTrack() в Timeline
        revision++
    }
}
```

```
}  
  
}
```

ПОТОК ДАННЫХ

USER ДЕЙСТВИЕ:

Drag video → Timeline



QML Track.onClipDropped



ClipManager.addClip(filepath, track, time)



cppTimeline.addClip() ← Вызов C++!



C++ TIMELINE:

1. Открыть файл через Ffmpeg
2. Получить длительность: 120.5 секунд
3. Создать TimelineClip
4. Добавить в m_clips
5. emit clipsChanged() ← Сигнал!



QML CONNECTIONS:

onClipsChanged() вызывается



Обновить UI



Клип появляется на timeline!

? ОТВЕТЫ НА ТВОИ ВОПРОСЫ

1. Почему указатель?

cpp

```
TimelineClip* getClip(int index);
```

БЕЗ указателя (копия):

cpp

```
TimelineClip getClip(int index) {  
    return m_clips[index]; // Копирует ВСЕ данные!  
}  
  
// QString filepath;    ← копируется  
// QMap effects;        ← копируется  
// double startTime;    ← копируется  
// ...и т.д.
```

С указателем:

```
cpp  
  
TimelineClip* getClip(int index) {  
    return &m_clips[index]; // Просто адрес! 8 байт  
}
```

Плюсы указателя:

- Быстро (не копирует данные)
- Можно изменить оригинал
- Можно вернуть nullptr если не найдено

2. Как работает FFmpeg?

```
cpp
```

```
double getClipSourceDuration(const QString& filepath) {
    // 1. Открыть файл
    AVFormatContext* ctx = nullptr;
    avformat_open_input(&ctx, filepath, ...);
    //      ↑
    // Указатель на указатель! FFmpeg создаст объект

    // 2. Прочитать метаданные
    avformat_find_stream_info(ctx, nullptr);

    // 3. Получить длительность
    double duration = ctx->duration / AV_TIME_BASE;
    //      ↑      ↑
    //      Микросекунды 1 миллион

    // 4. Закрыть
    avformat_close_input(&ctx);

    return duration; // В секундах!
}
```

Что происходит:

1. FFmpeg открывает файл `video.mp4`
2. Читает header файла (метаданные)
3. Находит информацию: "duration = 120500000 микросекунд"
4. Конвертирует: $120500000 / 1000000 = 120.5$ секунд
5. Возвращает 120.5

3. Это сложно для диплома?

НЕТ! Это НОРМАЛЬНО для диплома!

Твой уровень:

- ☒ Qt signals/slots
- ☒ FFmpeg integration
- ☒ QML ↔ C++ communication
- ☒ Timeline architecture

Это **хороший middle junior** уровень! Для диплома - **ОТЛИЧНО!**

Комиссия увидит:

- Понимание архитектуры

- Работу с библиотеками (FFmpeg)
- Интеграцию разных языков (C++/QML)
- Реальный работающий продукт

ЭТО РЕАЛЬНО КРУТО! 🔥

🔗 ИТОГО

1. **Timeline.cpp** = C++ бэкенд (данные + FFmpeg)
2. **main.cpp** = регистрация (`setContextProperty`)
3. **ClipManager.qml** = вызывает `cppTimeline.addClip()`
4. **Connections** = ловит сигналы (`onClipsChanged`)

ПРОСТО И МОЩНО!